Evaluation Report

Equipment and Resource Booking Management System

Computing: Software Development: Graded Unit 2

Stanislav Starishko

April 2026

Contents

Introduction 2

Requirements Coverage 2

- **Functional Requirements** 2
- **Non-Functional Requirements** 3
- **Business Rules** 3

Strengths and Weaknesses 4

- **Strengths** 4
- **Weaknesses** 5

Recommendations for Future Development 5

- ***Authentication and role-based access*** 5
- ***Persistent database*** 5
- ***Booking amendment*** 5
- ***Search and filtering on the bookings page*** 5
- ***Email notifications*** 6
- ***Pagination*** 6

Project Modifications 6

- ***Booking amendment descoped*** 6
- ***Authentication removed from scope*** 6
- ***User documentation approach changed*** 6
- ***BR-08 bug discovered and fixed*** 6
- ***Deployment to Render*** 6

Knowledge and Skills Gained 6

- ***Spring Boot and dependency injection*** 6
- ***JPA and Hibernate*** 7
- ***Thymeleaf*** 7
- ***Automated testing with JUnit 5 and Mockito*** 7
- ***Project management and scope control*** 7
- ***What I would do differently*** 7

Conclusion 7

Introduction

This document is the Evaluation Stage report for my Graded Unit 2 project: an Equipment and Resource Booking Management System. It covers the final stage of the project lifecycle, reflecting on what was built, how well it meets the original requirements, and what I would do differently with the benefit of hindsight.

The Planning Stage produced a detailed Action Plan Report with requirements, domain modelling and UML design. The Development Stage translated that design into a working Spring Boot web application with full test coverage and built-in user documentation. This report evaluates the finished product and the process behind it.

The finished application comprises 4 JPA entities, 4 services, 5 controllers, 64 automated tests.

Requirements Coverage

This section evaluates how the finished application maps to the requirements defined in the Planning Stage. Each functional requirement, non-functional requirement and business rule is assessed against the implemented system.

Functional Requirements

ID	Requirement	Status	Comment
FR-01	Register new equipment with name, category, location, condition and status	**Met**	Equipment form captures all specified fields. Category is selected from a dropdown populated by CategoryService.
FR-02	View, edit and delete equipment records	**Met**	Equipment list page with edit and delete actions. Deletion is blocked when active bookings exist (BR-07).
FR-03	Update equipment status	**Met**	Status field is editable through the equipment form. Automatic status updates are also handled by the booking logic (BR-08).
FR-04	Create a booking with equipment selection, date range and purpose	**Met**	Booking form with dropdown for available equipment, date pickers and purpose field. Only equipment with AVAILABLE or BOOKED status appears in the selection list (BR-01).
FR-05	Check for conflicts and prevent double-bookings	**Met**	A custom JPQL query in BookingRepository detects overlapping confirmed bookings. BookingService rejects the request with a BookingConflictException if a conflict is found (BR-02).
FR-06	Amend or cancel an existing booking	**Partially met**	Cancellation is fully implemented. Amendment (editing dates or equipment on an existing booking) was descoped during development; the decision was to focus on a reliable cancel-and-rebook workflow rather than risk introducing partial update bugs.
FR-07	Automatically update equipment status to reflect bookings	**Met**	BookingService sets equipment status to BOOKED on confirmation and reverts to AVAILABLE on cancellation when no other active bookings remain (BR-08).
FR-08	Search equipment by name, category, location or status	**Met**	Equipment list page includes a text search field and category/status filter dropdowns. Search queries are handled by EquipmentService methods.
FR-09	Search and filter bookings by date,	**Partially met**	Booking list displays all bookings with visible equipment and user columns. Dedicated search filters for date range and staff member were

ID	Requirement	Status	Comment
	equipment or staff member		not implemented as separate controls; the current list is manageable at the project's data scale but would need filtering for a production dataset.
FR-10	Dashboard with current and upcoming bookings	**Met**	Dashboard shows active bookings in a summary table with equipment name, user, dates and purpose.
FR-11	Summary of equipment availability across categories	**Met**	Dashboard displays availability counters (Available, Booked, Maintenance) as status cards at the top of the page.
FR-12	Create, edit and delete equipment categories	**Met**	Full CRUD for categories. Deletion is blocked when equipment is assigned (BR-06).

Non-Functional Requirements

ID	Area	Status	Comment
NFR-01	Usability	**Met**	Clean layout with consistent navigation, contextual help tooltips and a dedicated Help page with FAQ section. Manual testing confirmed that core tasks are completable without guidance.
NFR-02	Reliability	**Met**	Global exception handler catches all errors and displays user-friendly messages. Validation annotations prevent invalid data from reaching the service layer. No unhandled exceptions were observed during testing.
NFR-03	Performance	**Met**	Pages load within 2 seconds on localhost. On Render free tier the initial cold start takes longer due to JVM startup, but subsequent requests respond well within the threshold.
NFR-04	Maintainability	**Met**	Layered architecture (controller, service, repository) with consistent naming conventions, Javadoc on public methods and meaningful identifiers throughout.
NFR-05	Security	**Not met**	Basic authentication was not implemented. The system currently operates without login or role-based access. This was a conscious scope decision: implementing Spring Security would have added significant complexity without contributing to the core business logic that the project is assessed on. This is noted as a priority recommendation for future development.
NFR-06	Portability	**Met**	The application runs in any modern browser. No plugins or client-side installation required. Tested in Chrome and Firefox.

Business Rules

ID	Rule	Status	Comment
BR-01	Equipment under maintenance or unavailable cannot be booked	**Met**	EquipmentService.verifyAvailableForBooking() rejects MAINTENANCE and UNAVAILABLE statuses. Verified by 4 unit tests.

ID	Rule	Status	Comment
BR-02	Bookings for the same equipment must not overlap	**Met**	Custom JPQL query detects conflicts. Verified by repository integration tests and service-layer tests.
BR-03	Every booking must be linked to a registered staff member	**Met**	Booking entity has a mandatory ManyToOne relationship to User. Form dropdown only shows existing users.
BR-04	Every equipment item must belong to one category	**Met**	Equipment entity has a mandatory ManyToOne relationship to Category. Enforced at both database and form level.
BR-05	Booking end date must be equal to or later than start date	**Met**	InvalidBookingDatesException thrown when end date precedes start date. Verified by unit tests.
BR-06	Category deletion blocked if equipment is assigned	**Met**	CategoryService checks for assigned equipment before deletion. DeletionBlockedException thrown if items exist. Verified by 3 unit tests.
BR-07	Equipment deletion blocked if active bookings exist	**Met**	EquipmentService checks for confirmed bookings before deletion. Verified by 4 unit tests.
BR-08	Equipment status must reflect active bookings	**Met**	Status set to BOOKED on confirmation, reverted to AVAILABLE on cancellation when no other active bookings remain. Initially contained a bug (status not reverting) which was caught during manual testing and fixed with a regression test.

Strengths and Weaknesses

Strengths

The application delivers a fully working booking management system that covers all eight business rules defined in the Planning Stage. The core workflow, from registering equipment through to creating, confirming and cancelling bookings, operates exactly as designed. Every business rule is enforced at the service layer and verified by automated tests, which gives confidence that the logic is correct and will remain correct if the code is modified.

The **layered architecture** proved to be one of the strongest decisions in the project. Separating controllers, services and repositories meant that each layer could be developed and tested independently. Service-layer tests use Mockito mocks and run in milliseconds, while the repository integration test uses a real H2 database through @DataJpaTest. This separation also made the BR-08 bug fix straightforward: the change was isolated to a single method in BookingService without touching any controller or template code.

The **testing strategy** delivered real value. The deliberate combination of automated unit tests and manual browser testing caught a genuine defect (BR-08 status revert) that the unit tests alone did not detect. The structured approach of two documented test runs, with a regression test added between them, demonstrated that the testing process works as intended rather than being a box-ticking exercise.

Built-in user documentation was a better choice than a separate PDF manual. Help tooltips appear in context, directly on the pages where users need guidance, and the dedicated Help page with FAQ covers the scenarios most likely to confuse a first-time user. This approach keeps the documentation coupled to the application, so it cannot become separated or outdated.

The **code quality** is consistent throughout. Meaningful identifier names, Javadoc on public service methods, and a clear package structure make the codebase readable without excessive commenting. Even lambda parameters and loop variables have descriptive names rather than single letters, which is a small detail that makes a noticeable difference when reading the code.

The decision to **deploy the application to Render** was not part of the original plan but adds tangible value: anyone can access the working system without cloning the repository or installing Java locally.

Weaknesses

The most significant limitation is the **absence of authentication** (NFR-05). Any user who knows the URL can access the system and perform any action. In a real deployment this would be unacceptable, but for this project the trade-off was reasonable: Spring Security would have added a substantial layer of complexity without exercising any new business logic. It remains the single highest-priority item for future development.

The **H2 in-memory database** means all data is lost when the application restarts. The DataInitialiser repopulates sample data on startup, which is fine for development and demonstration, but would be unsuitable for any real use. Migrating to PostgreSQL or MySQL with persistent storage is a straightforward change since JPA abstracts the database layer, but it was not necessary for the scope of this project.

Booking amendment (FR-06) was only partially implemented. Users can cancel a booking but cannot edit its dates or equipment selection directly. The cancel-and-rebook workflow achieves the same outcome, but it is less convenient and creates a new booking record rather than preserving the original. This was a deliberate scope decision to avoid the complexity of partial updates with conflict re-validation, but it is an honest gap.

The **BR-08 bug** was caught during manual testing, not by the automated test suite. This highlights a weakness in the original test design: the unit test for cancelBooking only verified that the booking status changed to CANCELLED, without checking whether the equipment status was reverted. The gap existed because the test was written with Mockito mocks, which do not enforce the full interaction chain. A broader integration test would have caught it earlier.

Recommendations for Future Development

If this project were to move beyond the academic submission and into real use, the following changes would be my priorities, roughly in the order I would tackle them.

Authentication and role-based access is the most urgent gap. Spring Security with a form-based login would restrict access to authorised staff and allow the existing UserRole enum (ADMIN, STAFF) to control what each user can do. Admins could manage categories and users, while staff would be limited to equipment browsing and booking. The User entity already stores a password field and a role, so the data model is ready; only the security configuration and login flow need to be added.

Persistent database would be the second step. Replacing H2 with PostgreSQL requires changing a few lines in application.properties and adding the PostgreSQL driver dependency. JPA handles the rest. This would eliminate the data loss on restart and make the system viable for daily use. On Render, a managed PostgreSQL instance can be provisioned and connected to the existing service with minimal configuration.

Booking amendment deserves a proper implementation rather than the current cancel-and-rebook workaround. An edit form pre-populated with the existing booking details, with conflict re-validation on save, would give users a smoother experience. The main challenge is handling the case where the original equipment slot has already been taken by another booking during the edit, which requires careful transaction management.

Search and filtering on the bookings page (FR-09) should be completed. Date range pickers, an equipment dropdown and a staff member filter would make the page usable at scale. The service layer already supports the necessary queries; the missing piece is the controller logic and template markup to wire up the filter controls.

Email notifications would add practical value in a workplace setting. A confirmation email when a booking is created, a reminder before the booking start date, and a notification when a booking is cancelled by someone else would reduce the chance of missed handovers. Spring Boot's mail starter makes this relatively straightforward to integrate.

Pagination is not needed at the current data scale, but any list page would struggle with hundreds of records. Spring Data JPA supports paginated queries out of the box through Pageable parameters, so the change would be incremental rather than architectural.

Project Modifications

This section summarises the changes made to the original plan, design and implementation during the project, and how unforeseen issues were handled.

***Booking amendment descoped*.** The Planning Stage specified full amendment capability for existing bookings (FR-06). During implementation I decided to support cancellation only and rely on a cancel-and-rebook workflow. The reason was practical: editing a confirmed booking requires re-running conflict detection against the new dates while excluding the original booking from the overlap check. This added a layer of complexity that risked introducing subtle bugs in the most critical part of the system. The cancel-and-rebook approach achieves the same outcome using logic that was already built and tested. This was a conscious trade-off between feature completeness and reliability.

***Authentication removed from scope*.** The Planning Stage included NFR-05 requiring basic authentication. After reviewing the Development Stage marking criteria, I concluded that implementing Spring Security would consume significant time on configuration and session management without contributing to the business logic, testing or documentation categories where marks are awarded. The decision was to invest that time in thorough testing and built-in help features instead, which directly map to assessed criteria.

***User documentation approach changed*.** The original plan assumed a separate PDF user manual. During the testing phase (Phase 6), manual testing revealed specific points where users would benefit from in-context guidance. I decided to build help features directly into the application: tooltip icons on each page linking to the relevant Help section, and a dedicated Help page with section-by-section explanations and an FAQ. This keeps the documentation tied to the product rather than existing as a separate file that can become outdated.

***BR-08 bug discovered and fixed*.** This was the most significant unforeseen event during development. When manually testing scenario MT-04 (cancel booking and check equipment status), I found that cancelling the only active booking for an equipment item did not revert its status from BOOKED to AVAILABLE. The root cause was in BookingService.cancelBooking(): the method changed the booking status to CANCELLED but did not check whether other active bookings remained for the same equipment. The fix added a count query for remaining confirmed bookings and a conditional status revert. A regression test was added to BookingServiceTest, bringing the total from 63 to 64 automated tests. A second full test run confirmed the fix and verified no regressions.

***Deployment to Render*.** This was not part of the original plan but was added after development was complete. Deploying a Spring Boot application with an embedded H2 database to a cloud platform required creating a Dockerfile, optimising JVM memory settings for Render's free tier (512 MB RAM, 0.1 CPU), and adjusting startup parameters to ensure the application booted within the platform's port detection timeout. The live deployment is available at https://stan-equipment-rental-system.onrender.com and demonstrates the working system without requiring local setup.

Knowledge and Skills Gained

This project was my first experience building a complete web application from scratch using an enterprise framework. The HND course covered Java fundamentals, object-oriented programming and data structures, but everything related to Spring Boot, web development and automated testing was learned independently for this project. This section reflects on what I gained and what I would do differently.

Spring Boot and dependency injection changed how I think about structuring code. Before this project I would have wired objects together manually; constructor injection and the @Service/@Controller annotations enforce a discipline that makes the architecture almost self-documenting. Understanding how Spring manages the application

context was the steepest part of the learning curve, but once it clicked, everything from configuration to testing became more logical.

JPA and Hibernate taught me to think in terms of entities and relationships rather than raw SQL. Writing the custom JPQL conflict detection query for BR-02 was the most challenging single piece of code in the project. It required understanding how Hibernate translates JPQL to SQL, how parameter binding works, and how to test the query in isolation using @DataJpaTest with a real H2 instance. The experience of debugging a query that looked correct in Java but produced unexpected results in SQL was frustrating at the time but genuinely valuable.

Thymeleaf was new to me as a templating engine. The fragment-based layout system took some trial and error to get right, but once the shared header, navigation and footer fragments were in place, adding new pages became fast and consistent. Working with Thymeleaf also improved my understanding of how server-side rendering differs from client-side JavaScript frameworks.

Automated testing with JUnit 5 and Mockito built on the foundation I established in my previous project (ClydeConservation), where I first adopted @Nested grouping and @DisplayName annotations. What this project added was **mock-based isolation with Mockito**: writing tests where the service layer runs against fake dependencies rather than a real database. That shift forced me to think carefully about what each test actually verifies, and more importantly what it does not verify. The BR-08 experience was the clearest lesson: the cancelBooking test used mocks that confirmed the booking status changed to CANCELLED, but never checked whether the equipment status was reverted. A test that checks the wrong thing gives false confidence, and that is a mistake I will not repeat.

Project management and scope control was a less technical but equally important skill. Learning to descope features (booking amendment, authentication) based on assessment value rather than feature completeness was uncomfortable at first. My instinct was to build everything specified in the plan. But the project taught me that delivering a reliable, well-tested subset is worth more than a fragile, untested full feature set. This is a mindset shift I expect to apply in future work.

What I would do differently**.** If I were starting again, I would write integration tests from the beginning alongside the unit tests. The BR-08 bug could have been caught earlier with a single test that exercised the full cancel flow through the service layer with a real database. I would also plan the user documentation earlier rather than treating it as a final phase; the help content is better when it is written while the interface decisions are still fresh. Finally, I would set up the cloud deployment from day one rather than adding it at the end. Having a live environment throughout development would have caught environment-specific issues earlier and made progress visible at any point.

Conclusion

This project delivered a working Equipment and Resource Booking Management System that implements all eight business rules, passes 64 automated tests and includes built-in user documentation. Two functional requirements were partially met (FR-06 booking amendment, FR-09 booking filters) and one non-functional requirement was not met (NFR-05 authentication), all documented with clear rationale. The most valuable outcome was not the application itself but the experience of managing a full development lifecycle: from requirements through design, implementation, testing, bug fixing and deployment.

If I were to summarise the project in one sentence, it would be this: I learned more from the one bug my tests missed than from the sixty-three they caught.